HyPlan: An Open-Source Python Library for Planning Airborne Remote Sensing Campaigns

17 March 2026

Summary

Airborne remote sensing is a critical tool for Earth science, enabling observations at spatial and temporal scales that bridge ground-based measurements and satellite data. Airborne platforms carry imaging spectrometers, lidars, radars, and other instruments over study areas at altitudes ranging from a few hundred meters to over 20 kilometers, producing data products that support research in ecology, hydrology, atmospheric science, geology, and oceanography. Planning these campaigns, however, requires integrating a diverse set of constraints: aircraft performance envelopes, sensor geometry and ground sampling requirements, solar illumination conditions, terrain variability, cloud climatology, airport logistics, and coordination with satellite overpasses. Scientists typically address these constraints using ad hoc spreadsheets, manual calculations, and institutional knowledge, resulting in workflows that are error-prone, difficult to reproduce, and hard to transfer between campaigns or research groups.

HyPlan is an open-source Python library that provides a unified, reproducible framework for planning airborne remote sensing missions. It encodes the physics of sensor-platform-environment interactions into composable building blocks that can be combined to design, evaluate, and optimize flight campaigns. HyPlan supports the full mission planning lifecycle: from defining study areas and generating flight line patterns, through sensor swath and ground sample distance calculations, to computing complete mission plans with multi-day scheduling and flight line ordering optimization. The library ships with pre-configured models for more than a dozen NASA research instruments and 14 research aircraft, and provides tools for solar geometry analysis, terrain-aware swath modeling, cloud climatology assessment, airport selection, and satellite overpass prediction.

Statement of Need

Airborne science campaigns represent major investments of resources, often costing millions of dollars and requiring months of coordination among scientists,

pilots, instrument teams, and logistics personnel. A single deployment of a high-altitude aircraft such as the NASA ER-2 can cost tens of thousands of dollars per flight hour, and campaigns typically span weeks to months of field operations. Despite these costs, the flight planning process has remained largely manual and fragmented across disconnected tools.

Scientists planning airborne campaigns must simultaneously reason about several interacting domains:

Sensor performance. The ground sample distance (GSD) of a pushbroom imaging spectrometer depends on altitude above ground level, the sensor's instantaneous field of view, and the ratio of aircraft ground speed to sensor frame rate. For a given sensor, there exists a critical ground speed above which along-track pixels become undersampled. Swath width determines how many parallel flight lines are needed to cover a study area, and the required overlap between adjacent swaths depends on both the science requirements and the terrain relief, which causes swath width to vary along a flight line. Planning must account for these relationships to ensure that the collected data meets the spatial sampling requirements of the intended science application.

Solar geometry. Most passive optical and spectroscopic instruments require minimum solar elevation angles for adequate signal-to-noise ratios. For aquatic remote sensing, sun glint—specular reflection of sunlight from the water surface into the sensor's field of view—can contaminate observations or, in some applications, provide useful information about surface roughness (Cox and Munk 1954). The glint angle depends on solar position, sensor view geometry, and flight heading, and varies continuously along a flight line. Planning must identify time windows that satisfy both minimum illumination and glint avoidance (or targeting) constraints.

Logistics. Aircraft have finite endurance, and research aircraft often operate from airports with specific runway length or surface requirements. A campaign covering a large study area may require hundreds of flight lines that cannot be completed in a single sortie. The ordering of flight lines, selection of refueling airports, and allocation of lines to flight days all affect total campaign duration and cost. These combinatorial logistics problems interact with the time-varying solar constraints described above.

Satellite coordination. Many airborne campaigns are designed to collect coincident or near-coincident observations with satellite instruments for calibration, validation, or science synergy. Planning must predict satellite overpass times and ground-track geometry to identify windows of overlap.

No existing open-source tool addresses these questions in an integrated, programmatic manner. HyPlan fills this gap by providing a Python library that models the full chain of constraints relevant to airborne remote sensing mission planning, with validated geodesic calculations, physically consistent unit handling, and export to standard geospatial formats.

State of the Field

Several categories of tools address subsets of the airborne mission planning problem, but none provide an integrated, scriptable solution for science-driven flight planning.

Commercial flight planning software. Tools such as ForeFlight and Jeppesen FliteStar focus on pilot navigation, fuel planning, and regulatory compliance for general and commercial aviation. These tools are not designed to reason about scientific objectives such as ground sample distance, spectral coverage, solar geometry, or sensor-specific swath overlap requirements.

Geographic information systems. GIS platforms such as QGIS and ArcGIS can visualize flight lines and study areas, but lack domain-specific calculations for sensor modeling, aircraft performance, and mission timing. While scientists often use GIS tools to display planned flight tracks, the underlying planning calculations must be performed elsewhere.

UAS mission planners. In the unmanned aerial systems domain, tools such as Mission Planner and QGroundControl (Dronecode Project 2024) provide waypoint planning and autopilot integration. However, these are designed for small drones operating at low altitudes and short ranges, and do not model the sensor physics, solar geometry, or logistics constraints relevant to crewed research aircraft operating over large study areas at high altitudes.

Moving Lines. The most directly comparable tool is Moving Lines (LeBlanc 2018), a Python-based flight planning application developed for NASA airborne science campaigns. Moving Lines provides a graphical user interface combining an interactive map with spreadsheet-based waypoint editing and has been used operationally across numerous NASA field campaigns including ORACLES, IMPACTS, and ARCSIX. Moving Lines excels at real-time, interactive flight plan creation and modification during campaign operations, with features for overlaying satellite imagery, weather model output, and satellite ground tracks. However, Moving Lines is designed as a GUI application for manual, interactive planning rather than as a programmatic library. It does not provide sensor-specific swath and GSD calculations, terrain-aware analysis, automated flight line generation from study area polygons, or algorithmic flight line ordering optimization. HyPlan and Moving Lines are complementary: HyPlan addresses the pre-campaign science planning phase—determining how many flight lines are needed, what altitude and speed satisfy sensor requirements, when solar conditions permit data collection, and how to schedule lines across multiple flight days—while Moving Lines supports the tactical, day-of-flight planning and replanning that occurs during campaign execution.

Agency-internal tools. Various NASA centers and other research organizations have developed internal flight planning tools tailored to specific instruments or aircraft, but these are typically proprietary, undocumented, and tightly coupled to particular missions, making them difficult to adapt to new campaigns or share

across the research community.

HyPlan is, to our knowledge, the first open-source library that integrates sensor modeling, flight planning, environmental analysis, and logistics optimization for airborne remote sensing in a single, composable Python framework. Its programmatic API enables reproducible planning workflows that can be version-controlled, shared, and adapted across campaigns and research groups.

Software Design

HyPlan is organized into three functional groups—Flight Planning, Instruments, and Environment and Logistics—each containing several modules. The library is built on the scientific Python ecosystem, using NumPy (Harris et al. 2020) for numerical operations, pandas (McKinney 2010), GeoPandas (Jordahl et al. 2020), and Shapely (Gillies et al. 2007) for geospatial data structures, Pint (Grecco 2024) for physical units, Pydantic for data validation, and pymap3d (Hirsch 2018) and pyproj for coordinate transformations and geodesic calculations.

Several cross-cutting design decisions shape how these modules are implemented and used. All physical quantities carry explicit units via the Pint library (Grecco 2024), preventing the class of unit conversion errors that has historically caused failures in aerospace applications (NASA 1999) and supporting seamless mixing of aviation conventions (feet, knots, nautical miles) with scientific conventions (meters, m/s, kilometers). All distance and bearing calculations use Vincenty's inverse and direct formulae (Vincenty 1975) on the WGS84 ellipsoid via the `pymap3d` library, avoiding the errors inherent in spherical approximations which can exceed 0.3% for distances relevant to airborne surveys. Pydantic provides runtime validation of inputs such as geographic coordinates and flight line geometry, catching configuration errors early with descriptive error messages. Rather than providing a monolithic GUI application, HyPlan exposes its functionality through a composable Python API that can be used in scripts, Jupyter notebooks, and automated pipelines, enabling version-controlled, reproducible planning workflows. Results export to standard geospatial formats including GeoJSON, KML, shapefiles, and GeoDataFrames (via GeoPandas), and render as interactive Folium web maps for immediate visual feedback.

Flight Planning

The core abstraction in HyPlan is the `FlightLine`, which represents a geodesic flight segment defined by two geographic endpoints and an altitude above mean sea level. All distance and bearing calculations use Vincenty's formulae (Vincenty 1975) on the WGS84 ellipsoid, ensuring accuracy over the long baselines typical of airborne surveys. Flight lines support a rich set of geometric operations: clipping to polygon boundaries, splitting into segments of specified length, perpendicular and along-track offsetting, rotation around the midpoint, and reversal of flight direction. Each operation preserves the geodesic properties of the line and

maintains altitude and metadata. Flight lines can be exported to GeoJSON, KML, shapefiles, and GeoDataFrame representations for integration with GIS workflows.

The `FlightBox` module generates arrays of parallel flight lines covering arbitrary study areas. Given a sensor, altitude, and desired cross-track overlap, the module computes the swath spacing and distributes lines symmetrically around a center line or across a study polygon. Lines can be oriented along a user-specified azimuth or automatically aligned with the minimum rotated bounding rectangle of the study area. An alternating-direction (boustrophedon) option produces a serpentine pattern that minimizes transit time between adjacent lines. When a clipping polygon is provided, lines are extended by a configurable buffer before clipping to ensure full coverage of the study area boundary.

Complete mission plans are computed by the `flight_plan` module, which models each phase of a sortie: takeoff and initial climb from the departure airport, transit to the first flight line, data collection segments along each flight line, transit between lines, descent, approach, and landing at the destination airport. Each phase uses the aircraft's altitude-dependent speed profile, and the module computes segment-by-segment timing, distance, heading, and altitude. The output is a GeoDataFrame containing the full flight trajectory with segment classification, which can be visualized as altitude profiles and interactive maps.

For campaigns with many flight lines, the `flight_optimizer` module formulates line ordering as a graph problem. Nodes in a NetworkX (Hagberg et al. 2008) directed graph represent airport locations and flight line endpoints; edges are weighted by transit time, computed from Dubins path distances and aircraft performance models including climb and descent. The optimizer applies a greedy nearest-neighbor heuristic subject to constraints on aircraft endurance, maximum daily flight time, and the availability of refueling airports. When endurance is exhausted, the optimizer identifies the nearest airport from which at least one additional unvisited flight line is reachable, routes the aircraft for refueling, and continues. The result is a multi-day schedule with ordered flight lines, refueling stops, and per-day flight time accounting. While the greedy algorithm does not guarantee global optimality, it produces practical schedules efficiently and can serve as a starting point for further manual refinement.

Realistic aircraft maneuvering between waypoints is modeled using Dubins curves (Dubins 1957; Walker 2011), which compute the shortest path between two oriented points (position plus heading) subject to a minimum turning radius constraint. The turning radius is derived from the aircraft's speed and maximum bank angle as $R = v^2/(g \tan \phi)$, where v is true airspeed, g is gravitational acceleration, and ϕ is the bank angle. Dubins paths consist of combinations of circular arcs and straight segments and provide realistic estimates of transit distance and time, particularly for the tight reversals required between adjacent, oppositely directed flight lines.

Instruments

HyPlan provides sensor models for the major instrument types used in airborne remote sensing, each implemented as a class with methods for computing ground-level observing geometry as a function of flight altitude and speed.

Imaging spectrometers are modeled as line scanners (pushbroom or whiskbroom) characterized by their total cross-track field of view, number of across-track pixels, and frame rate. The nadir ground sample distance is computed as $\mathrm{GSD} = 2h\tan(\mathrm{IFOV}/2)$, where h is the altitude above ground level and IFOV is the instantaneous field of view per pixel. The swath width is $W = 2h\tan(\mathrm{FOV}/2)$, where FOV is the total field of view. The critical ground speed—the maximum speed at which along-track pixels remain contiguous—is $v_{\mathrm{crit}} = \mathrm{GSD}/(T \cdot s)$, where T is the frame period and s is the along-track sampling factor. Pre-configured models are provided for NASA instruments including AVIRIS-3 (Thompson et al. 2022), AVIRIS-5, HyTES (Johnson et al. 2011), PRISM (Mouroulis et al. 2014), MASTER, GLiHT (VNIR, thermal, and SIF channels), GCAS, eMAS, and PICARD, with instrument-specific parameters drawn from published specifications.

Frame cameras are modeled with two-dimensional sensor arrays, computing ground footprint dimensions, GSD, and along-track sampling interval from the focal length, sensor dimensions, pixel count, and frame rate. This model supports planning for photogrammetric and mapping cameras commonly flown alongside spectroscopic instruments.

Lidar is represented by the LVIS (Land, Vegetation, and Ice Sensor) full-waveform scanning lidar (Blair et al. 1999), which uses a conical scan pattern to map a swath beneath the aircraft. The LVIS model includes three standard lens configurations (narrow, medium, and wide divergence) that trade off footprint size against the ability to achieve contiguous spatial coverage. The effective swath width depends on whether the laser pulse rate is sufficient to tile the geometric swath without gaps at the given footprint size and aircraft speed: $W_{\mathrm{eff}} = \min(W_{\mathrm{geom}}, d^2 f/v)$, where d is the footprint diameter, f is the pulse repetition rate, and v is the ground speed.

Synthetic aperture radar is modeled for the UAVSAR instrument (Hensley et al. 2008) in its L-band, P-band (AirMOSS), and Ka-band (GLISTIN-A) configurations. Unlike nadir-looking optical sensors, side-looking radar illuminates a swath offset from the flight track, defined by near-range and far-range incidence angles. The model computes slant-range resolution from bandwidth ($\Delta r = c/2B$), ground-range resolution as a function of incidence angle, swath width, ground distance to swath edges, and interferometric line spacing for repeat-pass or cross-track interferometry applications.

Terrain-aware swath polygons are generated by the `swath` module, which combines flight line geometry with sensor models and digital elevation data. For each point along a flight line, the module traces rays from the sensor to the

ground at the port and starboard half-angles, computing the intersection with the terrain surface. The resulting swath polygon accounts for terrain-induced variations in swath width and position, which can be substantial in mountainous terrain. Swath width statistics (minimum, mean, maximum) are computed from the polygon geometry.

Environment and Logistics

Solar position calculations use the `sunposition` library (Reda and Andreas 2004) to compute solar azimuth and elevation at arbitrary locations and times. The `sun` module provides functions to determine data collection windows by finding the times at which solar elevation crosses specified thresholds (e.g., 20° or 30° minimum elevation), sampled at one-minute resolution across a range of dates. This supports seasonal planning by identifying how data collection windows shift throughout the year and across latitude.

Sun glint prediction is implemented in the `glint` module, which computes the specular reflection angle between the sun and each pixel in the sensor's cross-track field of view. The glint angle θ_g is computed from the solar zenith angle θ_s, view zenith angle θ_v, and relative azimuth $\Delta\phi$ as:

$$\cos\theta_g = \cos\theta_v \cos\theta_s - \sin\theta_v \sin\theta_s \cos\Delta\phi$$

The module evaluates this expression along the full length of a flight line at configurable along-track spacing, across the full cross-track field of view at 1° increments, producing a spatially resolved map of glint angles. This enables identification of flight headings and time windows that minimize (or, for glint-targeting applications, maximize) specular contamination. The line-of-sight from sensor to ground target is computed using `pymap3d`'s spheroid intersection, accounting for Earth curvature and sensor altitude.

Terrain analysis uses 30-meter Copernicus DEM data, downloaded automatically from AWS and cached locally with spatial indexing via R-tree. The terrain module supports bulk elevation queries, DEM tile merging via GDAL (GDAL/OGR contributors 2024), and a vectorized ray-terrain intersection algorithm. For each observer position (sensor location along a flight line), the algorithm computes the slant range to the WGS84 ellipsoid, determines a search window bounded by the minimum and maximum terrain elevations, samples points along the ray at a configurable precision (default 10 m), queries the DEM at each sample point, and identifies the first intersection where the terrain surface exceeds the ray altitude. This approach enables computation of actual ground intersection points for off-nadir sensor view angles, which is essential for accurate swath polygon generation over complex terrain.

Cloud climatology is estimated from the MODIS Terra and Aqua instruments using Google Earth Engine (Gorelick et al. 2017). The `clouds` module extracts binary cloud masks from the MOD09GA and MYD09GA quality assurance

bands, computes daily spatial cloud fraction over user-defined polygons, and aggregates statistics across years to estimate clear-sky probability by day of year. A campaign simulation function models the process of visiting multiple study sites subject to clear-sky thresholds, consecutive-day limits, and weekend exclusions, enabling estimation of the number of campaign days required to achieve complete coverage with acceptable cloud contamination.

Aircraft performance is modeled for 14 research aircraft spanning the range of platforms used in airborne Earth science: from high-altitude jets (NASA ER-2, WB-57) through medium-altitude turboprops (Gulfstream III, Gulfstream IV, King Air B200, Twin Otter, P-3 Orion) to smaller platforms (Cessna Caravan, Dynamic Aviation B200). Each aircraft model specifies service ceiling, cruise speed, best rate of climb at sea level, rate of climb at service ceiling, descent rate, approach speed, endurance, range, and maximum bank angle. Cruise speed varies with altitude, modeled either as a linear interpolation between a low-altitude speed and the service-ceiling speed, or as a user-defined piecewise profile. The rate of climb follows a linear model $\mathrm{ROC}(h) = \mathrm{ROC}_{\mathrm{ceil}} + (\mathrm{ROC}_{\mathrm{sl}} - \mathrm{ROC}_{\mathrm{ceil}})(1 - h/C)$, where C is the service ceiling, yielding an analytical solution for climb time:

$$t = \frac{C}{\mathrm{ROC}_{\mathrm{sl}} - \mathrm{ROC}_{\mathrm{ceil}}} \ln \frac{\mathrm{ROC}(h_0)}{\mathrm{ROC}(h_1)}$$

This produces a realistic exponential climb profile where the rate of climb decreases with altitude. Descent is modeled at a constant rate. The approach phase follows an IFR (Instrument Flight Rules) profile with an intermediate fix and final approach fix at standard distances from the runway.

Airport selection leverages the OurAirports global database, providing search and filtering by geographic proximity, country, airport type (large, medium, small), minimum runway length, and runway surface type. The `Airport` class exposes runway dimensions, headings, surface types, and elevation, supporting automated identification of airports that meet the operational requirements of a given aircraft.

Satellite overpass prediction supports coordination of airborne observations with 15 satellite missions including PACE, Landsat-8, Landsat-9, Sentinel-2A/B, Sentinel-3A/B, JPSS-1 (NOAA-20), JPSS-2 (NOAA-21), Aqua, Terra, ICESat-2, CALIPSO, CloudSat, and EarthCARE. The module fetches two-line element (TLE) sets from CelesTrak, propagates satellite orbits using the Skyfield library (Rhodes 2019), computes sub-satellite ground tracks, and generates swath footprint polygons by offsetting the ground track perpendicular to the flight direction by half the instrument swath width. A two-pass overpass finding algorithm first performs a coarse scan to identify candidate time windows, then refines at high temporal resolution within those windows. Overpasses are segmented into individual passes, filtered by solar zenith angle for daytime usability, and tested for spatial overlap with study area polygons.

Research Impact Statement

Early versions of HyPlan have been used to support flight planning for NASA airborne science campaigns including BioSCape (Cardoso et al. 2025), SHIFT (Chadwick et al. 2025), and the 2022–2023 ABoVE AVIRIS campaigns (Miller et al. 2025), where it addresses the pre-campaign science planning phase: determining spatial coverage requirements, computing sensor performance at candidate altitudes, evaluating solar timing constraints, and producing optimized multi-day flight schedules. The library's programmatic approach enables planning workflows that can be documented, version-controlled, and shared across campaign teams, improving reproducibility and institutional knowledge transfer compared to traditional spreadsheet-based approaches.

The library is documented with a full API reference hosted on GitHub Pages and 18 Jupyter notebooks that serve as both tutorials and integration tests. The notebooks cover every module and demonstrate realistic workflows including multi-sensor campaigns, terrain-aware swath analysis, solar timing optimization, and multi-day flight scheduling. A dedicated validation notebook verifies HyPlan's core calculations against independent reference values: geodesic distances are compared with Vincenty's published test cases, solar position calculations are checked against NOAA's solar calculator, and sensor GSD and swath width calculations are validated against analytical solutions.

HyPlan is designed to be extensible to new aircraft, sensors, and mission configurations. Adding a new imaging spectrometer requires only specifying the field of view, pixel count, and frame rate; adding a new aircraft requires specifying the performance envelope parameters. This extensibility positions HyPlan to support the broader airborne science community beyond the NASA instruments and aircraft currently included.

AI Usage Disclosure

Generative AI tools (Claude) were used to assist with drafting and editing this manuscript. The software itself was developed with AI coding assistance (Claude and ChatGPT). All AI-generated content was reviewed and verified by the author.

Acknowledgements

The author thanks Samuel LeBlanc for developing Moving Lines and for contributions to the airborne science planning community that informed HyPlan's design.

This work was supported by the National Aeronautics and Space Administration.

References

Blair, J. Bryan, David L. Rabine, and Michelle A. Hofton. 1999. "The Laser Vegetation Imaging Sensor: A Medium-Altitude, Digitisation-Only, Airborne Laser Altimeter for Mapping Vegetation and Topography." *ISPRS Journal of Photogrammetry and Remote Sensing* 54 (2–3): 115–22. https://doi.org/10.1016/S0924-2716(99)00002-7.

Cardoso, Ana W., Erin L. Hestir, Jasper A. Slingsby, et al. 2025. "The Biodiversity Survey of the Cape (BioSCape), Integrating Remote Sensing with Biodiversity Science." *Npj Biodiversity* 4 (1): 5. https://doi.org/10.1038/s44185-024-00071-5.

Chadwick, K. Dana, Kerry Cawse-Nicholson, David R. Thompson, et al. 2025. "Unlocking Ecological Insights from Sub-Seasonal Visible-to-Shortwave Infrared Imaging Spectroscopy: The SHIFT Campaign." *Ecosphere* 16 (3): e70194. https://doi.org/10.1002/ecs2.70194.

Cox, Charles, and Walter Munk. 1954. "Measurement of the Roughness of the Sea Surface from Photographs of the Sun's Glitter." *Journal of the Optical Society of America* 44 (11): 838–50. https://doi.org/10.1364/JOSA.44.000838.

Dronecode Project. 2024. *QGroundControl: Intuitive and Powerful Ground Control Station for the MAVLink Protocol*. http://qgroundcontrol.com.

Dubins, Lester E. 1957. "On Curves of Minimal Length with a Constraint on Average Curvature, and with Prescribed Initial and Terminal Positions and Tangents." *American Journal of Mathematics* 79 (3): 497–516. https://doi.org/10.2307/2372560.

GDAL/OGR contributors. 2024. *GDAL/OGR Geospatial Data Abstraction Software Library*. Open Source Geospatial Foundation, released. https://doi.org/10.5281/zenodo.5884351.

Gillies, Sean et al. 2007. *Shapely: Manipulation and Analysis of Geometric Objects*. Zenodo, released. https://doi.org/10.5281/zenodo.5597138.

Gorelick, Noel, Matt Hancher, Mike Dixon, Simon Ilyushchenko, David Thau, and Rebecca Moore. 2017. "Google Earth Engine: Planetary-Scale Geospatial Analysis for Everyone." *Remote Sensing of Environment* 202: 18–27. https://doi.org/10.1016/j.rse.2017.06.031.

Grecco, Hernan E. 2024. *Pint: Operate and Manipulate Physical Quantities in Python*. https://pint.readthedocs.io.

Hagberg, Aric A., Daniel A. Schult, and Pieter J. Swart. 2008. "Exploring Network Structure, Dynamics, and Function Using NetworkX." *Proceedings of the 7th Python in Science Conference*, 11–15. https://doi.org/10.25080/TCWV9851.

Harris, Charles R., K. Jarrod Millman, Stéfan J. van der Walt, et al. 2020. "Array Programming with NumPy." *Nature* 585 (7825): 357–62. https://doi.org/10.1038/s41586-020-2649-2.

Hensley, Scott, Kevin Wheeler, Gregory Sadowy, et al. 2008. "Status of a UAVSAR Designed for Repeat Pass Interferometry for Deformation Measurements." *IEEE Aerospace Conference Proceedings*, 1–8. https://doi.org/10.1109/AERO.2008.4526385.

Hirsch, Michael. 2018. "PyMap3D: 3-D Coordinate Conversions for Terrestrial and Geospace Environments." *Journal of Open Source Software* 3 (23): 580. https://doi.org/10.21105/joss.00580.

Johnson, William R., Simon J. Hook, Michael Foote, and Pantazis Z. Mouroulis. 2011. "HyTES: Thermal Emission Spectrometer on a New Generation Airborne Platform." *Proceedings of SPIE* 8154: 81540V. https://doi.org/10.1117/12.894361.

Jordahl, Kelsey, Joris Van den Bossche, Martin Fleischmann, et al. 2020. *GeoPandas: Python Tools for Geographic Data*. Zenodo, released. https://doi.org/10.5281/zenodo.3946761.

LeBlanc, Samuel E. 2018. *Moving Lines: NASA Airborne Research Flight Planning Tool*. Zenodo, released. https://doi.org/10.5281/zenodo.1478126.

McKinney, Wes. 2010. "Data Structures for Statistical Computing in Python." *Proceedings of the 9th Python in Science Conference*, 56–61. https://doi.org/10.25080/Majora-92bf1922-00a.

Miller, Charles E., Robert O. Green, David R. Thompson, et al. 2025. "Airborne Imaging Spectroscopy Surveys of Arctic and Boreal Alaska and Northwestern Canada 2017–2023." *Scientific Data* 12: 692. https://doi.org/10.1038/s41597-025-04898-w.

Mouroulis, Pantazis, Byron E. Van Gorp, Robert O. Green, et al. 2014. "Portable Remote Imaging Spectrometer Coastal Ocean Sensor: Design, Characteristics, and First Flight Results." *Applied Optics* 53 (7): 1363–80. https://doi.org/10.1364/AO.53.001363.

NASA. 1999. *Mars Climate Orbiter Mishap Investigation Board Phase I Report*. https://llis.nasa.gov/llis_lib/pdf/1009464main1_0641-mr.pdf.

Reda, Ibrahim, and Afshin Andreas. 2004. "Solar Position Algorithm for Solar Radiation Applications." *Solar Energy* 76 (5): 577–89. https://doi.org/10.1016/j.solener.2003.12.003.

Rhodes, Brandon. 2019. *Skyfield: High Precision Research-Grade Positions for Planets and Earth Satellites Generator*. https://rhodesmill.org/skyfield/.

Thompson, David R., Ian Brosnan, Bo-Cai Gao, et al. 2022. "AVIRIS-3: Next-Generation Airborne Visible Infrared Imaging Spectrometer." *2022 IEEE Aerospace Conference (AERO)*, 1–7. https://doi.org/10.1109/AERO53065.2022.9843565.

Vincenty, Thaddeus. 1975. "Direct and Inverse Solutions of Geodesics on the Ellipsoid with Application of Nested Equations." *Survey Review* 23 (176): 88–93. https://doi.org/10.1179/sre.1975.23.176.88.

Walker, Andrew. 2011. "Hard Real-Time Motion Planning for Autonomous Vehicles." PhD thesis, Swinburne University of Technology.